

[Realtime Chat & Notification System](#)

[Features](#)

[Tech Stack](#)

[Architecture Overview](#)

[Prerequisites](#)

[Setup Instructions](#)

[Environment Variables](#)

[Redis Configuration](#)

[BullMQ Setup](#)

[Socket.IO Events Reference](#)

[REST API Documentation](#)

[Database Schema \(Prisma\)](#)

[Project Structure](#)

[Known Limitations / Future Improvements](#)

[Author](#)

Realtime Chat & Notification System

A production-style real-time chat and notification platform built with **NestJS**, **Socket.IO**, **Redis**, **BullMQ**, **PostgreSQL/Prisma**, and **Next.js**. Built as a Week 4 internship deliverable covering WebSockets, background job processing, and event-driven architecture.

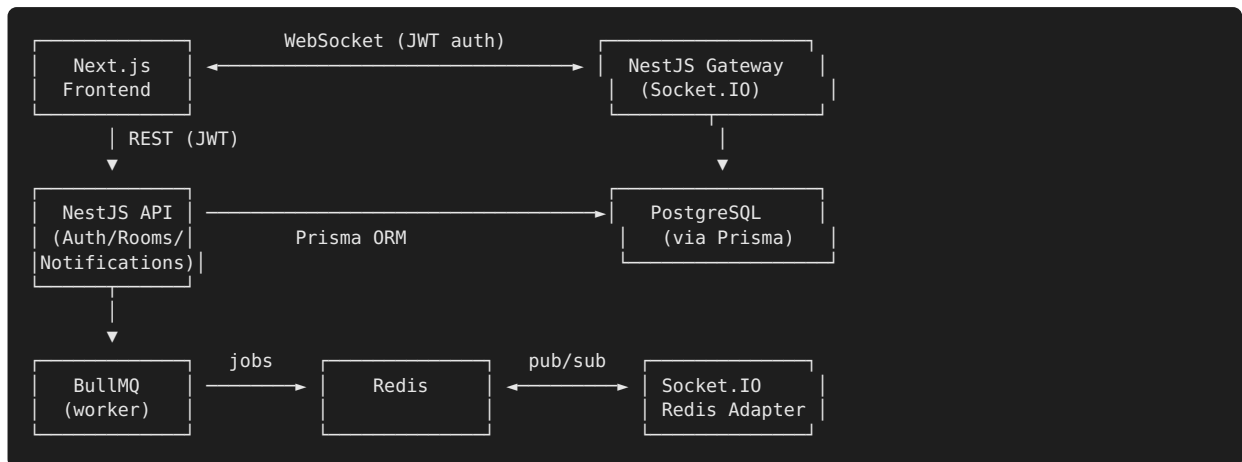
Features

- 🔒 JWT authentication (signup/login)
- 💬 Real-time private messaging and group chat rooms
- 📺 Live typing indicators
- 🌐 Online/offline presence tracking
- 🔔 In-app notification center with unread badges and toast popups
- 📧 Background job processing for notifications via BullMQ
- 📡 Redis Pub/Sub for horizontally scalable Socket.IO
- 🔍 Live user search (find people by name/email, not by exposing IDs)
- 📱 Fully responsive UI (mobile + desktop)

Tech Stack

Layer	Technology
Backend framework	NestJS
Real-time	Socket.IO
Database	PostgreSQL
ORM	Prisma (v7, driver adapters)
Cache / Pub-Sub / Queue broker	Redis (via Docker)
Background jobs	BullMQ
Frontend framework	Next.js 15 (App Router)
Styling	Tailwind CSS

Architecture Overview



- **Socket.IO Gateway** handles all real-time events (messages, typing, presence) and is protected by a custom JWT guard that validates the token on the WebSocket handshake.
- **Redis Pub/Sub** (via `@socket.io/redis-adapter`) lets Socket.IO events reach a user regardless of which backend instance their socket is attached to — required for horizontal scaling.
- **BullMQ** decouples notification creation from the request/response cycle. When a message is sent, a job is queued instead of writing to the DB inline, and a separate worker (`NotificationsProcessor`) processes it asynchronously.

Prerequisites

- Node.js 18+
- PostgreSQL (running locally or remote)
- Docker Desktop (for Redis)
- npm

Setup Instructions

1. Clone and install

```
git clone <repo-url>
cd realtime-chat-notification-system
```

2. Start Redis (via Docker)

```
docker run -d --name redis-chat -p 6379:6379 --restart unless-stopped redis:7-alpine
```

Verify it's running:

```
docker ps
```

3. Backend setup

```
cd backend
```

```
npm install
```

Create `backend/.env` :

```
DATABASE_URL="postgresql://<db_user>:<db_password>@localhost:5432/realtime_chat_db?schema=public"
JWT_SECRET="replace-with-a-long-random-string"
```

Create the database (via pgAdmin or `psql`), then run migrations:

```
npx prisma migrate dev --name init
```

Start the backend:

```
npm run start:dev
```

Backend runs on **http://localhost:3000**.

4. Frontend setup

```
cd frontend
npm install
```

Create `frontend/.env.local` :

```
NEXT_PUBLIC_API_URL=http://localhost:3000
```

Start the frontend:

```
npm run dev
```

Frontend runs on **http://localhost:3001** (Next.js auto-selects the next free port since 3000 is taken by the backend).

Environment Variables

Backend (`backend/.env`)

Variable	Description	Example
<code>DATABASE_URL</code>	PostgreSQL connection string	<code>postgresql://postgres:pass@localhost:5432/realtime_chat_db</code>
<code>JWT_SECRET</code>	Secret used to sign/verify JWTs	any long random string

Frontend (`frontend/.env.local`)

Variable	Description	Example
<code>NEXT_PUBLIC_API_URL</code>	Base URL of the backend API/socket server	<code>http://localhost:3000</code>

Redis Configuration

Redis serves two purposes in this system:

1. **BullMQ broker** — stores the notification job queue. Configured in `app.module.ts` :

```
BullModule.forRoot({
  connection: { host: 'localhost', port: 6379 },
})
```

2. **Socket.IO Pub/Sub adapter** — configured in `main.ts` via `@socket.io/redis-adapter` , allowing WebSocket events to broadcast across multiple backend instances.

Redis runs in a Docker container (`redis-chat`) exposed on the default port `6379` . No password/auth configured for local development — **do not deploy this configuration as-is to production**; add `requirepass` and TLS for any public deployment.

BullMQ Setup

- **Queue name:** `notifications`
- **Producer:** `ChatGateway` — every time a message is sent, a `new-message` job is queued for each other room member.
- **Consumer:** `NotificationsProcessor` (`@Processor('notifications')`) — picks up jobs, writes the notification to PostgreSQL, and returns.

This decouples notification writes from the real-time message-send path, so a slow DB write never blocks message delivery.

Socket.IO Events Reference

Client → Server

Event	Payload	Description
<code>room:join</code>	<code>{ chatRoomId }</code>	Join a room to receive its live messages
<code>room:leave</code>	<code>{ chatRoomId }</code>	Leave a room
<code>typing:start</code>	<code>{ chatRoomId }</code>	Broadcast that the user started typing
<code>typing:stop</code>	<code>{ chatRoomId }</code>	Broadcast that the user stopped typing
<code>message:send</code>	<code>{ chatRoomId, content }</code>	Send a message (persists + broadcasts + queues notifications)

Server → Client

Event	Payload	Description
<code>message:new</code>	<code>Message</code> object	A new message was posted to a joined room
<code>typing:update</code>	<code>{ userId, chatRoomId, isTyping }</code>	Someone's typing status changed
<code>user:online</code>	<code>{ userId }</code>	A user came online
<code>user:offline</code>	<code>{ userId }</code>	A user went offline
<code>notification:new</code>	<code>{ type, title, body, metadata }</code>	A new notification was created for this user

Authentication

The JWT is passed in the Socket.IO handshake:

```
const socket = io(SOCKET_URL, { auth: { token: accessToken } });
```

The `ChatGateway.handleConnection` method validates it before allowing the connection; the `WsJwtGuard` additionally protects individual event handlers.

REST API Documentation

Base URL: `http://localhost:3000`

Auth

Method	Endpoint	Body	Description
POST	<code>/auth/signup</code>	<code>{ name, email, password }</code>	Create an account, returns <code>{ accessToken, user }</code>
POST	<code>/auth/login</code>	<code>{ email, password }</code>	Log in, returns <code>{ accessToken, user }</code>

Users

Method	Endpoint	Query	Description	Auth
GET	<code>/users/search</code>	<code>?q=<string></code>	Search users by name/email (excludes self)	✓

Rooms

Method	Endpoint	Body	Description	Auth
POST	<code>/rooms/direct</code>	<code>{ otherUserId }</code>	Create or fetch an existing 1-on-1 room	✓
POST	<code>/rooms/group</code>	<code>{ name, memberIds[] }</code>	Create a group room	✓
GET	<code>/rooms</code>	—	List all rooms the user belongs to	✓
GET	<code>/rooms/:id/messages</code>	—	Get message history for a room	✓

Notifications

Method	Endpoint	Description	Auth
GET	<code>/notifications</code>	List all notifications for the user	✓
GET	<code>/notifications/unread-count</code>	Get unread count	✓
PATCH	<code>/notifications/:id/read</code>	Mark one notification as read	✓
PATCH	<code>/notifications/read-all</code>	Mark all as read	✓

All protected routes require:

```
Authorization: Bearer <accessToken>
```

Database Schema (Prisma)

Model	Purpose
User	Account info, online status, last seen
ChatRoom	Direct (isGroup: false) or group chat container
ChatRoomMember	Join table: user ↔ room, tracks typing state
Message	Persisted chat messages
Notification	Persisted notifications, read/unread flag, metadata
UserConnection	Tracks active socket sessions per user (multi-device support)

See `backend/prisma/schema.prisma` for full field definitions and relations.

Project Structure

```
realtime-chat-notification-system/
├── backend/
│   ├── prisma/
│   │   └── schema.prisma
│   └── src/
│       ├── auth/           # JWT auth, guards, strategy
│       ├── chat/           # Socket.IO gateway
│       ├── rooms/          # Room REST API
│       ├── notifications/  # Notifications REST API + BullMQ processor
│       ├── users/          # User search
│       └── prisma/         # Prisma service
├── frontend/
│   └── src/
│       ├── app/
│       │   ├── login/
│       │   ├── signup/
│       │   └── chat/
│       ├── components/    # NotificationBell, ToastContainer
│       ├── context/       # Auth, Socket, Notification contexts
│       └── lib/           # API client, socket client
```

Known Limitations / Future Improvements

- New rooms created by another user don't appear in your sidebar until refresh (no live "room created" broadcast yet)
- No read receipts, file/image sharing, or emoji reactions yet (see Bonus Challenge items)
- No rate limiting on message sending
- Redis container runs without auth — fine for local dev, needs hardening for production

Author

Nimra Talpur — Full-Stack Development Intern, DawoodTech NextGen